

Instituto Tecnológico José Mario Molina Pasquel Henríquez.

Alumnos:

ALVAREZ DONOSA FABIAN MISAEL

NAVA MARTINEZ GAEL SALVADOR

PADILLA GUTIERREZ OSVALDO

DE ANDA GUTIERREZ ISMAEL DE JESUS

PEREZ CORONADO VICTOR HUGO

Docente: Andres Salomon Enriquez Ochoa

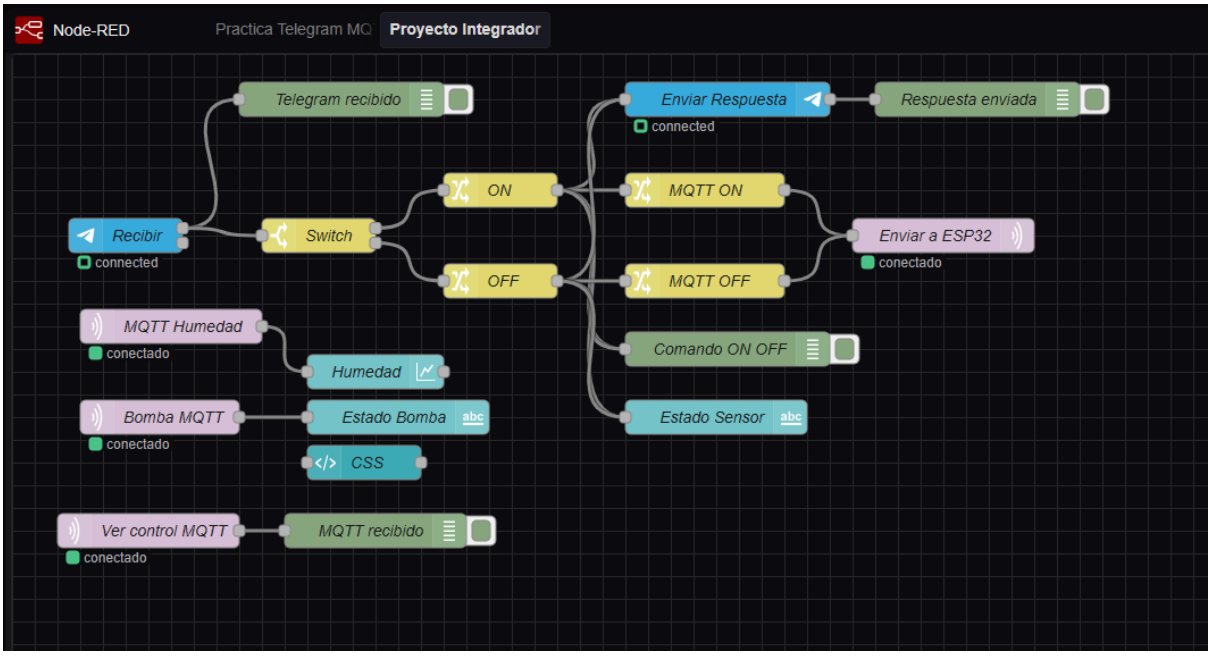
Materia: Internet de las cosas.

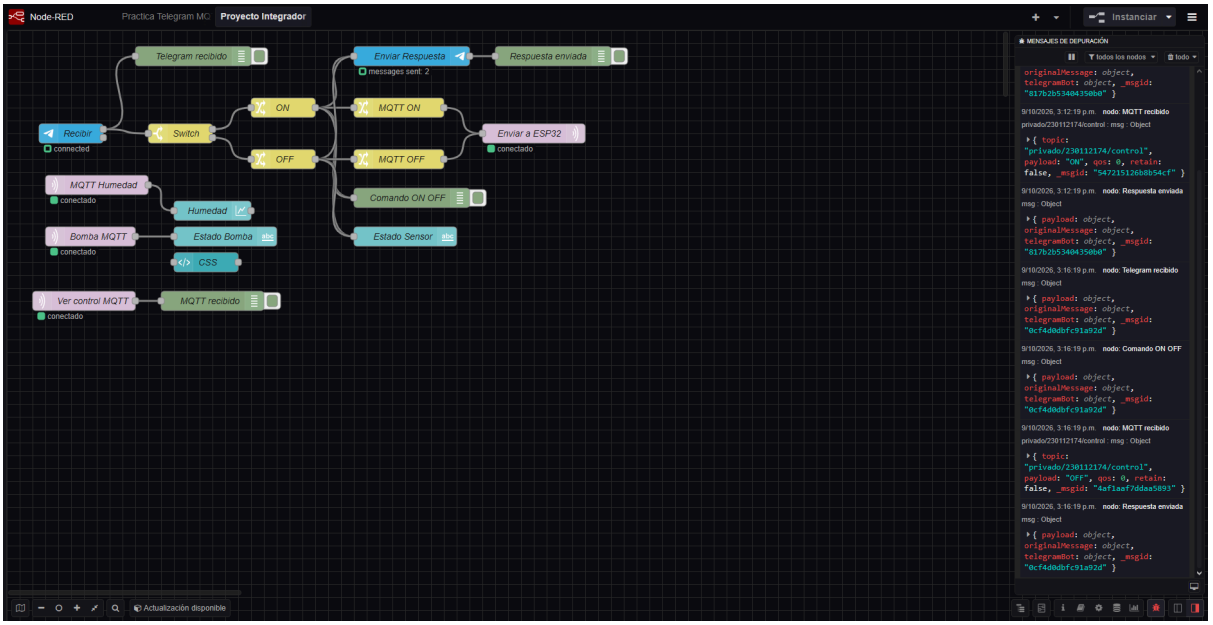
Proyecto Integrador U2.

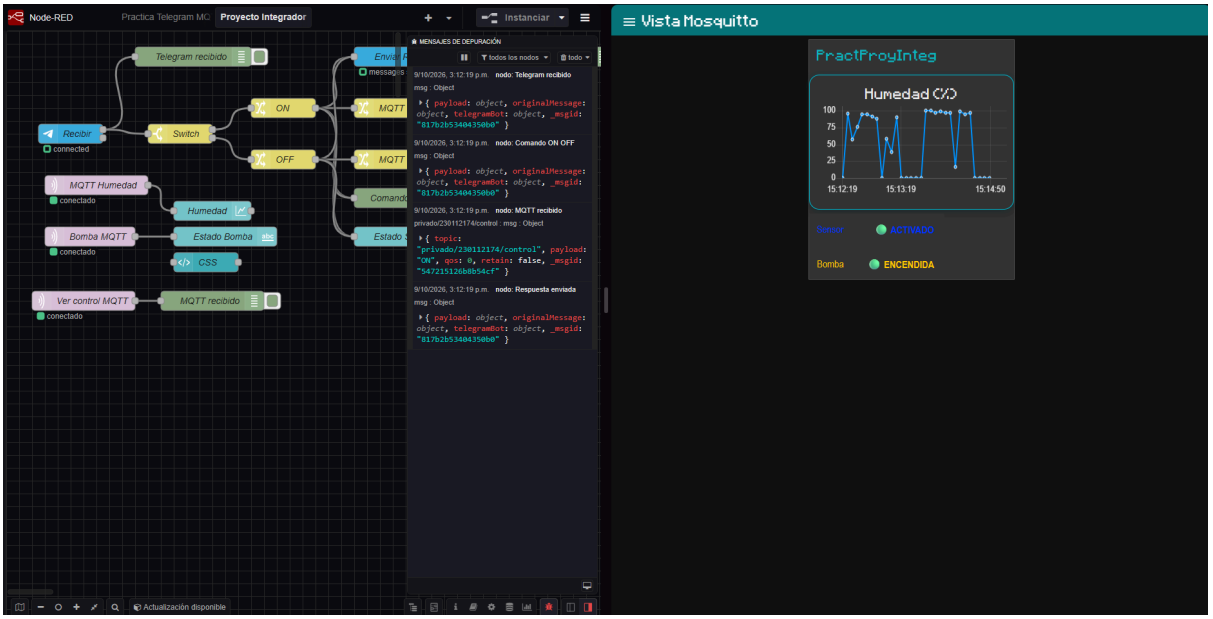
Fecha de entrega:09/10/2026.

Diagrama Json
Bot Telegram
Diagrama Electronico
Configuración Servidor
Pruebas
Preguntas
Links
Video
Repositorio

Diagrama Json



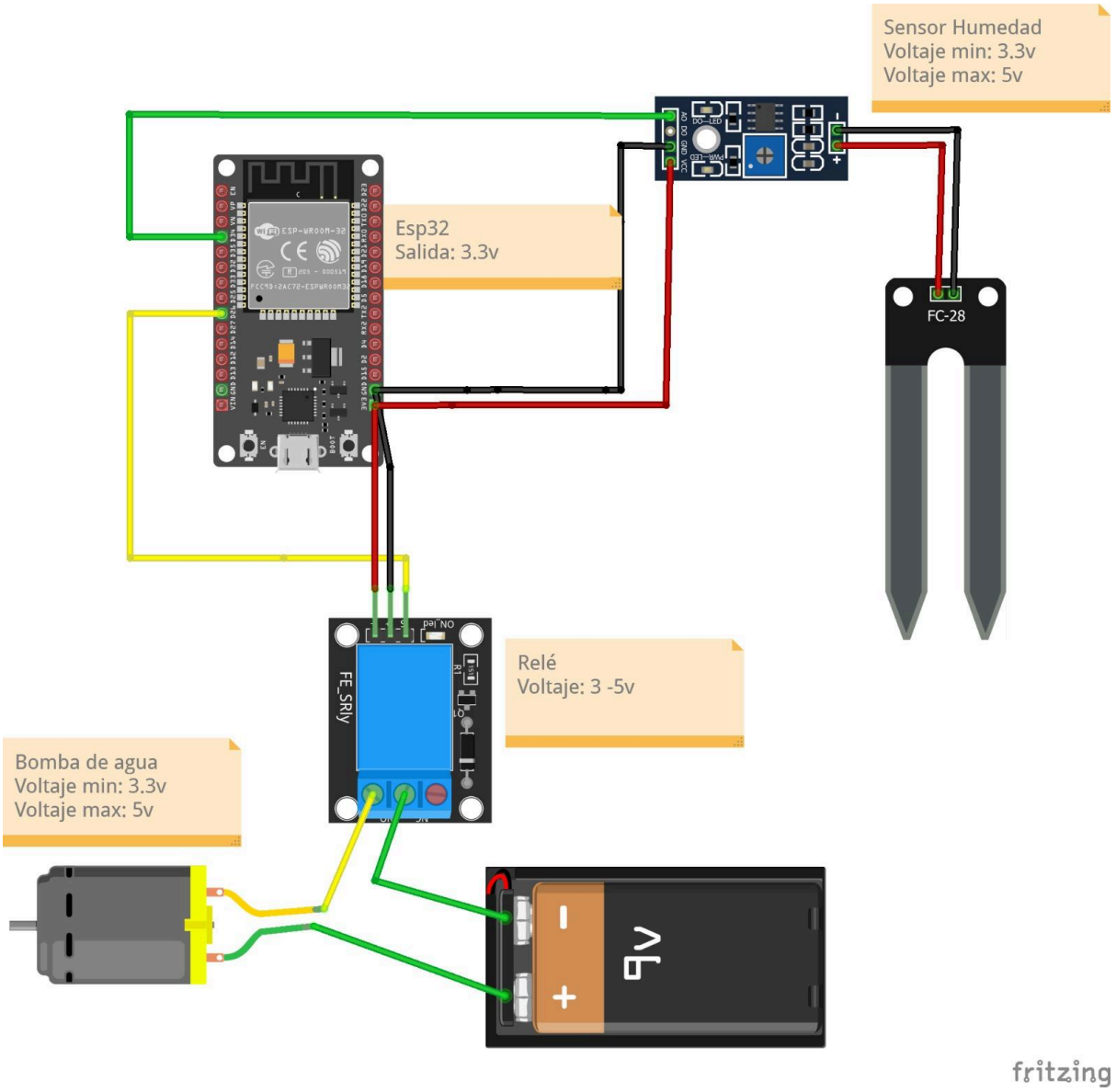



Node-RED
Practica Telegram MC
Proyecto Integrador
Instanciar
Telegram recibido
Recibir
connected
Switch
ON
OFF
MQTT
MQTT Humedad
conectado
Humedad
Bomba MQTT
conectado
Estado Bomba
CSS
Ver control MQTT
conectado
MQTT recibido
MENSAJES DE DEPURACIÓN
todos los nodos
todo
9/10/2026, 3:12:19 p.m. nodo: Telegram recibido
msg : Object
{ payload: object, originalMessage: object, telegramBot: object, _msgid: "817b2b53404350b0" }
9/10/2026, 3:12:19 p.m. nodo: Comando ON OFF
msg : Object
{ payload: object, originalMessage: object, telegramBot: object, _msgid: "817b2b53404350b0" }
9/10/2026, 3:12:19 p.m. nodo: MQTT recibido
privado/230112174/control : msg : Object
{ topic: "privado/230112174/control", payload: "ON", qos: 0, retain: false, _msgid: "547215126b8b54cf" }
9/10/2026, 3:12:19 p.m. nodo: Respuesta enviada
msg : Object
{ payload: object, originalMessage: object, telegramBot: object, _msgid: "817b2b53404350b0" }
Actualización disponible
Vista Mosquitto
PractProyInteg
Humedad (%)
100
75
50
25
0
15:12:19
15:13:19
15:14:50
Sensor
ACTIVADO
Bomba
ENCENDIDA

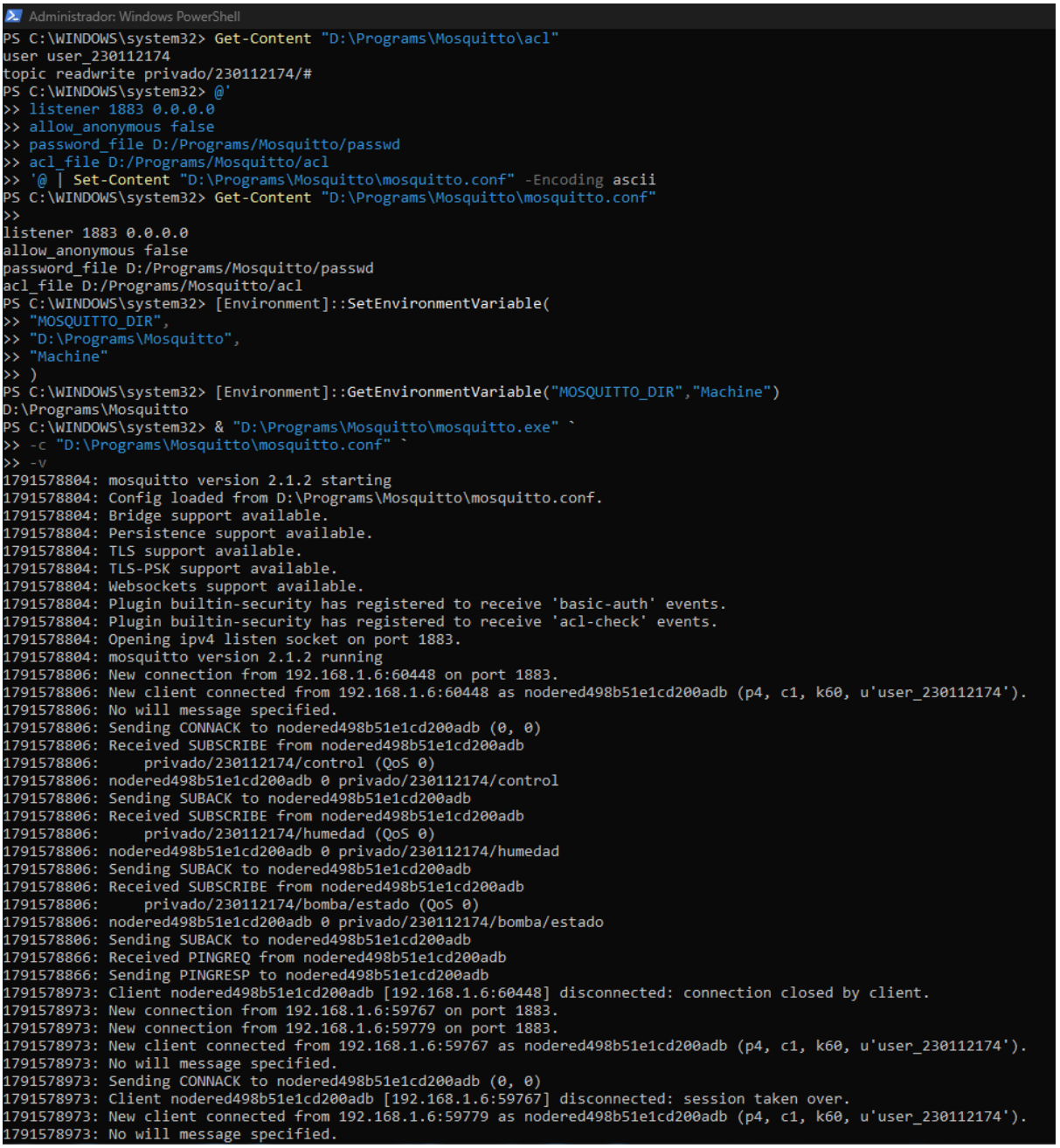
Bot Telegram



Diagrama Electronico



Configuración Servidor



Seleccionar Administrador: Windows PowerShell

```
PS C:\WINDOWS\system32> @'
>> from cryptography import x509
>> from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID
>> from cryptography.hazmat.primitives import hashes, serialization
>> from cryptography.hazmat.primitives.asymmetric import rsa
>> from datetime import datetime, timedelta
>> import ipaddress
>> import pathlib
>> import sys
>>
>> if len(sys.argv) != 2:
>>     print("Uso: generar_certificados.py <IP>")
>>     sys.exit(1)
>>
>> server_ip = sys.argv[1]
>>
>> carpeta = pathlib.Path(r"D:\Programs\Mosquitto\certs")
>> carpeta.mkdir(parents=True, exist_ok=True)
>>
>> # ==============================
>> # CLAVE PRIVADA DE LA CA
>> # ==============================
>> ca_key = rsa.generate_private_key(
>>     public_exponent=65537,
>>     key_size=2048
>> )
>>
>> ca_name = x509.Name([
>>     x509.NameAttribute(NameOID.COUNTRY_NAME, "MX"),
>>     x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Proyecto 6 MQTT"),
>>     x509.NameAttribute(NameOID.COMMON_NAME, "Proyecto6-CA"),
>> ])
>>
>> ca_cert = (
>>     x509.CertificateBuilder()
>>     .subject_name(ca_name)
>>     .issuer_name(ca_name)
>>     .public_key(ca_key.public_key())
>>     .serial_number(x509.random_serial_number())
>>     .not_valid_before(datetime.utcnow() - timedelta(days=1))
>>     .not_valid_after(datetime.utcnow() + timedelta(days=3650))
>>     .add_extension(
>>         x509.BasicConstraints(ca=True, path_length=None),
>>         critical=True
>>     )
>>     .add_extension(
>>         x509.KeyUsage(
>>             digital_signature=True,
>>             content_commitment=False,
>>             key_encipherment=False,
>>             data_encipherment=False,
>>             key_agreement=False,
>>             key_cert_sign=True,
>>             crl_sign=True,
>>             encipher_only=False,
>>             decipher_only=False,
>>         ),
>>         critical=True
>>     )
>>     .sign(ca_key, hashes.SHA256())
>> )
>>
>> # ==============================
```

Seleccionar Administrador: Windows PowerShell

```
>> )
>>
>> # ==============================
>> # CLAVE PRIVADA DEL SERVIDOR
>> # ==============================
>> server_key = rsa.generate_private_key(
>>     public_exponent=65537,
>>     key_size=2048
>> )
>>
>> server_name = x509.Name([
>>     x509.NameAttribute(NameOID.COUNTRY_NAME, "MX"),
>>     x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Proyecto 6 MQTT"),
>>     x509.NameAttribute(NameOID.COMMON_NAME, server_ip),
>> ])
>>
>> server_cert = (
>>     x509.CertificateBuilder()
>>     .subject_name(server_name)
>>     .issuer_name(ca_name)
>>     .public_key(server_key.public_key())
>>     .serial_number(x509.random_serial_number())
>>     .not_valid_before(datetime.utcnow() - timedelta(days=1))
>>     .not_valid_after(datetime.utcnow() + timedelta(days=825))
>>     .add_extension(
>>         x509.SubjectAlternativeName([
>>             x509.IPAddress(ipaddress.ip_address(server_ip))
>>         ]),
>>         critical=False
>>     )
>>     .add_extension(
>>         x509.BasicConstraints(ca=False, path_length=None),
>>         critical=True
>>     )
>>     .add_extension(
>>         x509.ExtendedKeyUsage([
>>             ExtendedKeyUsageOID.SERVER_AUTH
>>         ]),
>>         critical=False
>>     )
>>     .sign(ca_key, hashes.SHA256())
>> )
>>
>> # ==============================
>> # GUARDAR ARCHIVOS
>> # ==============================
>>
>> (carpeta / "ca.key").write_bytes(
>>     ca_key.private_bytes(
>>         encoding=serialization.Encoding.PEM,
>>         format=serialization.PrivateFormat.TraditionalOpenSSL,
>>         encryption_algorithm=serialization.NoEncryption()
>>     )
>> )
>>
>> (carpeta / "ca.crt").write_bytes(
>>     ca_cert.public_bytes(serialization.Encoding.PEM)
>> )
>>
>> (carpeta / "server.key").write_bytes(
>>     server_key.private_bytes(
>>         encoding=serialization.Encoding.PEM,
>>         format=serialization.PrivateFormat.TraditionalOpenSSL,
```

```
Seleccionar Administrador: Windows PowerShell
>>
>> (carpeta / "server.crt").write_bytes(
>>     server_cert.public_bytes(serialization.Encoding.PEM)
>> )
>>
>> print()
>> print("CERTIFICADOS GENERADOS CORRECTAMENTE")
>> print("IP incluida en el certificado:", server_ip)
>> print()
>> print("Archivos:")
>> print(carpeta / "ca.crt")
>> print(carpeta / "ca.key")
>> print(carpeta / "server.crt")
>> print(carpeta / "server.key")
>> '@ | Set-Content "D:\Programs\Mosquitto\generar_certificados.py" -Encoding UTF8
PS C:\WINDOWS\system32> py -3.10 "D:\Programs\Mosquitto\generar_certificados.py" $ip

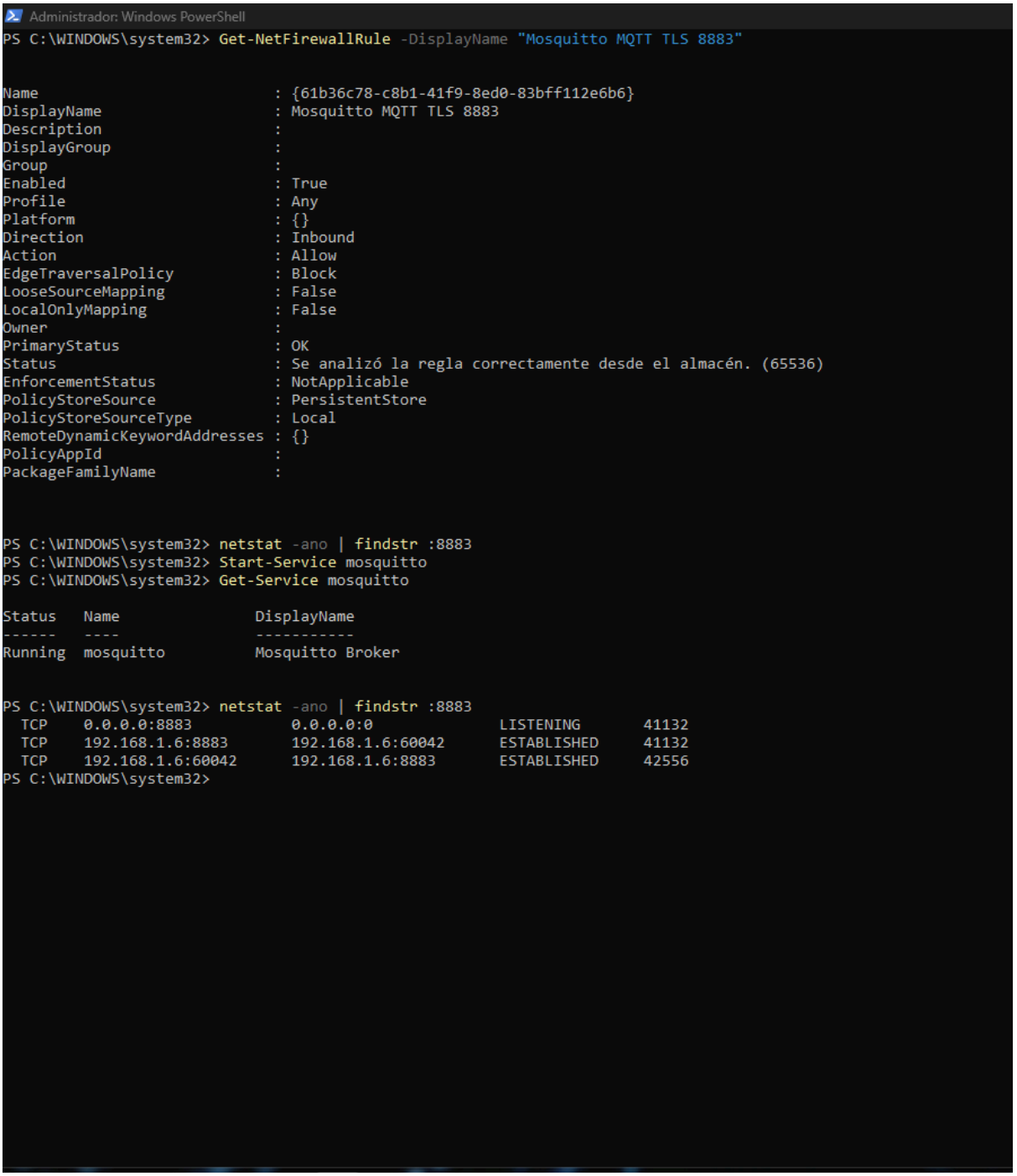
CERTIFICADOS GENERADOS CORRECTAMENTE
IP incluida en el certificado: 192.168.1.4

Archivos:
D:\Programs\Mosquitto\certs\ca.crt
D:\Programs\Mosquitto\certs\ca.key
D:\Programs\Mosquitto\certs\server.crt
D:\Programs\Mosquitto\certs\server.key
PS C:\WINDOWS\system32> Get-ChildItem "D:\Programs\Mosquitto\certs"


    Directorio: D:\Programs\Mosquitto\certs


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----     09/10/2026  06:08 p. m.           1164 ca.crt
-a----     09/10/2026  06:08 p. m.           1675 ca.key
-a----     09/10/2026  06:08 p. m.           1188 server.crt
-a----     09/10/2026  06:08 p. m.           1679 server.key


PS C:\WINDOWS\system32> Copy-Item "D:\Programs\Mosquitto\mosquitto.conf" `
>> "D:\Programs\Mosquitto\mosquitto.conf.backup" -Force
PS C:\WINDOWS\system32> notepad "D:\Programs\Mosquitto\mosquitto.conf"
PS C:\WINDOWS\system32> Stop-Service mosquitto -ErrorAction SilentlyContinue
PS C:\WINDOWS\system32> & "D:\Programs\Mosquitto\mosquitto.exe" `
>> -c "D:\Programs\Mosquitto\mosquitto.conf" `
>> -v
1791591053: mosquitto version 2.1.2 starting
1791591053: Config loaded from D:\Programs\Mosquitto\mosquitto.conf.
1791591053: Bridge support available.
1791591053: Persistence support available.
1791591053: TLS support available.
1791591053: TLS-PSK support available.
1791591053: Websockets support available.
1791591053: Plugin builtin-security has registered to receive 'basic-auth' events.
1791591053: Plugin builtin-security has registered to receive 'acl-check' events.
1791591053: Opening ipv4 listen socket on port 1883.
1791591053: Opening ipv4 listen socket on port 8883.
1791591053: mosquitto version 2.1.2 running
1791591157: mosquitto version 2.1.2 terminating
PS C:\WINDOWS\system32>
```


Administrador: Windows PowerShell
PS C:\WINDOWS\system32> Get-NetFirewallRule -DisplayName "Mosquitto MQTT TLS 8883"
Name : {61b36c78-c8b1-41f9-8ed0-83bff112e6b6}
DisplayName : Mosquitto MQTT TLS 8883
Description :
DisplayGroup :
Group :
Enabled : True
Profile : Any
Platform : {}
Direction : Inbound
Action : Allow
EdgeTraversalPolicy : Block
LooseSourceMapping : False
LocalOnlyMapping : False
Owner :
PrimaryStatus : OK
Status : Se analizó la regla correctamente desde el almacén. (65536)
EnforcementStatus : NotApplicable
PolicyStoreSource : PersistentStore
PolicyStoreSourceType : Local
RemoteDynamicKeywordAddresses : {}
PolicyAppId :
PackageFamilyName :
PS C:\WINDOWS\system32> netstat -ano | findstr :8883
PS C:\WINDOWS\system32> Start-Service mosquitto
PS C:\WINDOWS\system32> Get-Service mosquitto
Status Name DisplayName
Running mosquitto Mosquitto Broker
PS C:\WINDOWS\system32> netstat -ano | findstr :8883
TCP 0.0.0.0:8883 0.0.0.0:0 LISTENING 41132
TCP 192.168.1.6:8883 192.168.1.6:60042 ESTABLISHED 41132
TCP 192.168.1.6:60042 192.168.1.6:8883 ESTABLISHED 42556
PS C:\WINDOWS\system32>

```
Seleccionar Administrador: Windows PowerShell
PS C:\WINDOWS\system32> py -3.10 "D:\Programs\Mosquitto\generar_certificados.py" 192.168.1.6

CERTIFICADOS GENERADOS CORRECTAMENTE
IP incluida en el certificado: 192.168.1.6

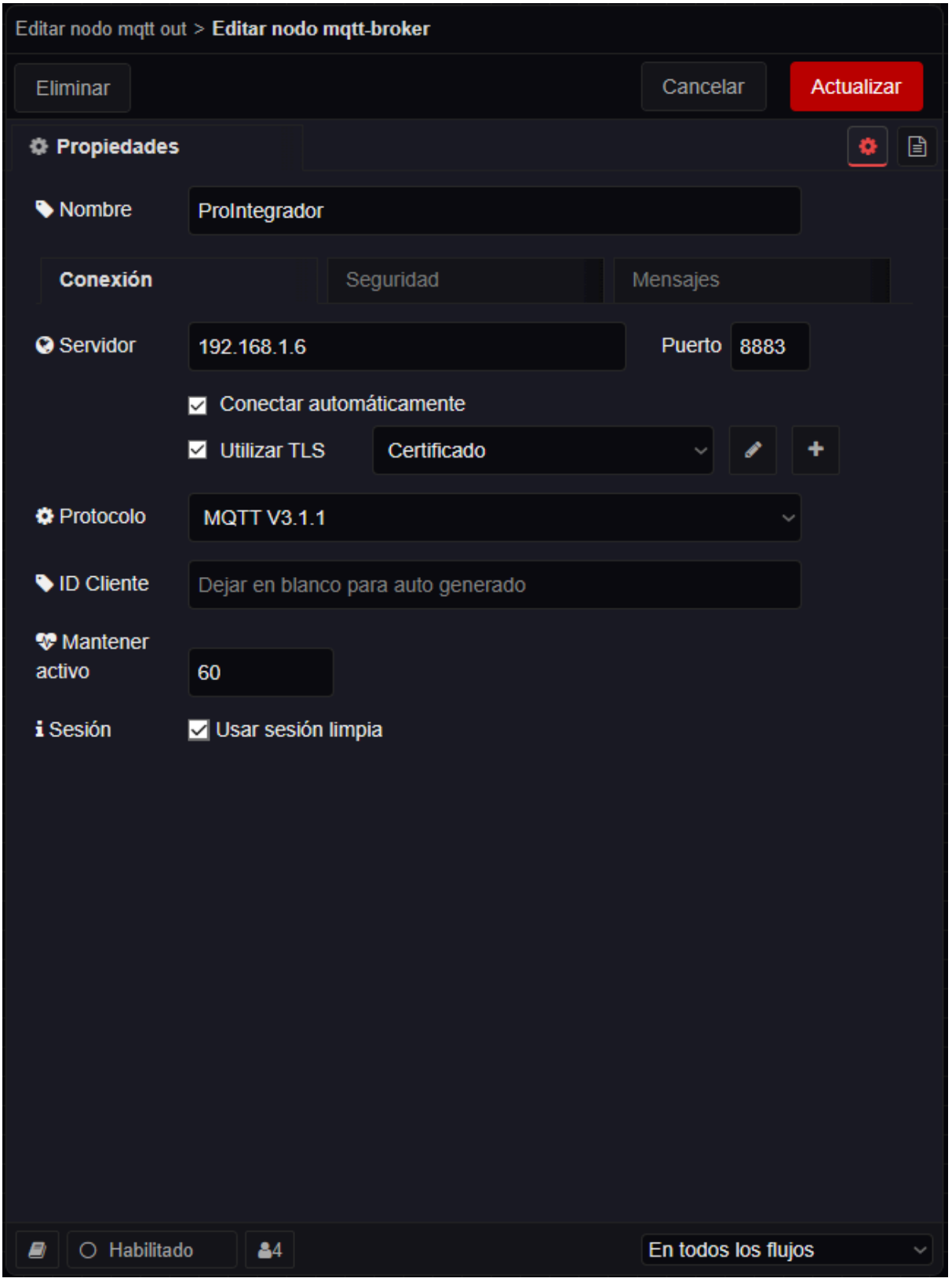
Archivos:
D:\Programs\Mosquitto\certs\ca.crt
D:\Programs\Mosquitto\certs\ca.key
D:\Programs\Mosquitto\certs\server.crt
D:\Programs\Mosquitto\certs\server.key
PS C:\WINDOWS\system32> & "D:\Programs\Mosquitto\mosquitto.exe" `
>> -c "D:\Programs\Mosquitto\mosquitto.conf" `
>> -v
1791591976: mosquitto version 2.1.2 starting
1791591976: Config loaded from D:\Programs\Mosquitto\mosquitto.conf.
1791591976: Bridge support available.
1791591976: Persistence support available.
1791591976: TLS support available.
1791591976: TLS-PSK support available.
1791591976: Websockets support available.
1791591976: Plugin builtin-security has registered to receive 'basic-auth' events.
1791591976: Plugin builtin-security has registered to receive 'acl-check' events.
1791591976: Opening ipv4 listen socket on port 1883.
1791591976: Opening ipv4 listen socket on port 8883.
1791591976: mosquitto version 2.1.2 running
1791591976: New connection from 192.168.1.4:57585 on port 8883.
1791591977: OpenSSL Error while trying to get the error[0]: error:0A000126:SSL routines::unexpected eof while reading
1791591977: Client 192.168.1.4 [192.168.1.4:57585] disconnected: protocol error.
1791591984: mosquitto version 2.1.2 terminating
PS C:\WINDOWS\system32> New-NetFirewallRule `
>> -DisplayName "Mosquitto MQTT TLS 8883" `
>> -Direction Inbound `
>> -Protocol TCP `
>> -LocalPort 8883 `
>> -Action Allow

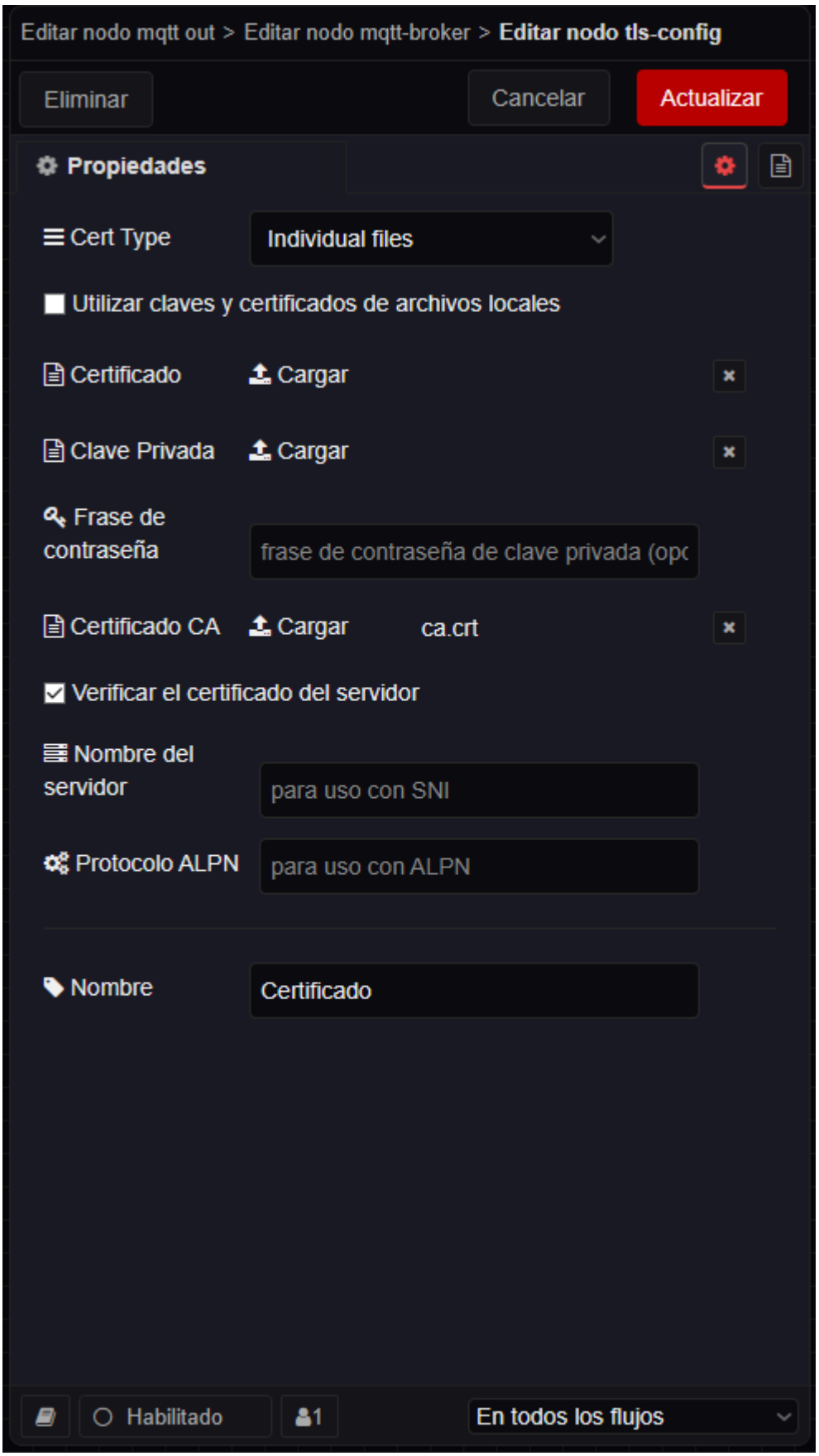

Name                          : {61b36c78-c8b1-41f9-8ed0-83bff112e6b6}
DisplayName                   : Mosquitto MQTT TLS 8883
Description                   :
DisplayGroup                  :
Group                         :
Enabled                       : True
Profile                       : Any
Platform                      : {}
Direction                     : Inbound
Action                        : Allow
EdgeTraversalPolicy           : Block
LooseSourceMapping            : False
LocalOnlyMapping              : False
Owner                         :
PrimaryStatus                 : OK
Status                        : Se analizó la regla correctamente desde el almacén. (65536)
EnforcementStatus             : NotApplicable
PolicyStoreSource             : PersistentStore
PolicyStoreSourceType         : Local
RemoteDynamicKeywordAddresses : {}
PolicyAppId                   :
PackageFamilyName             :

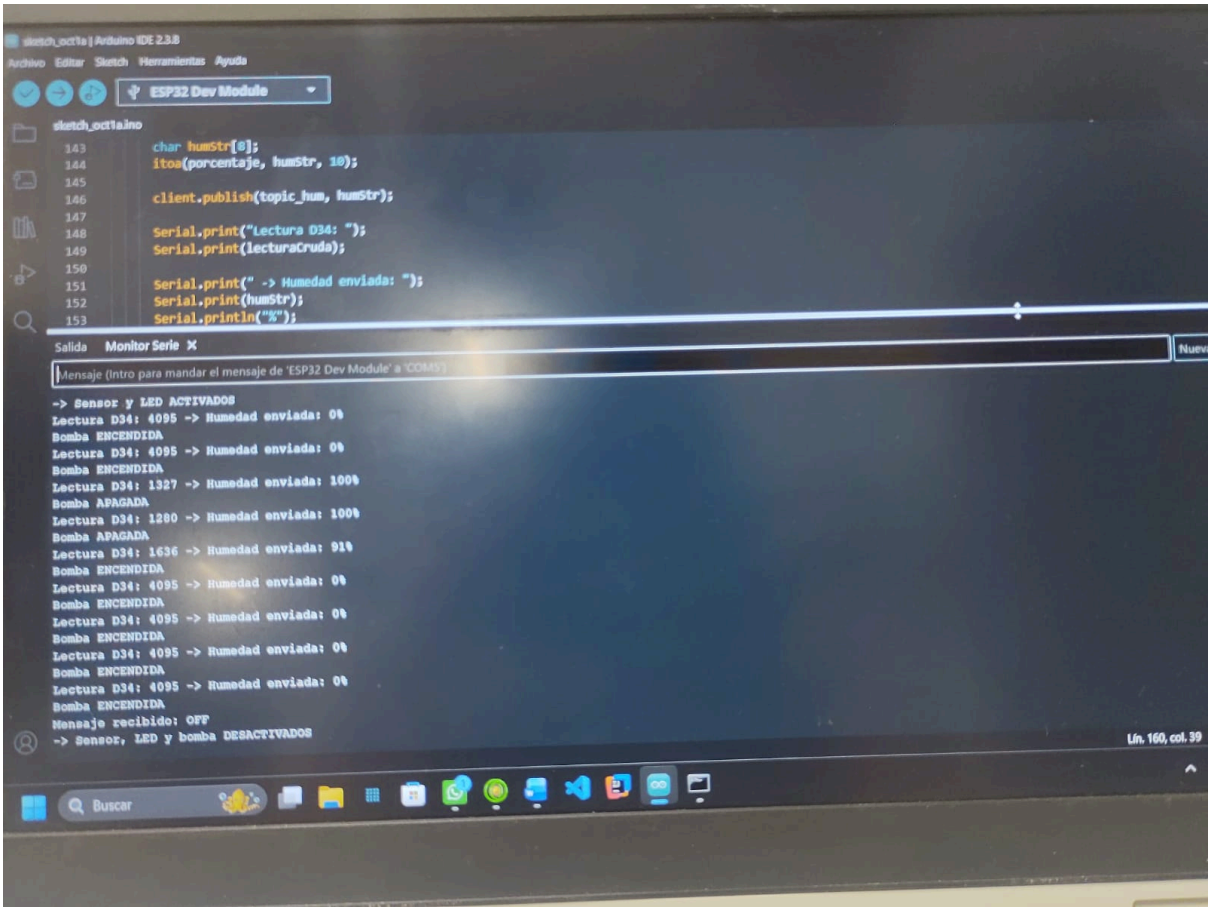


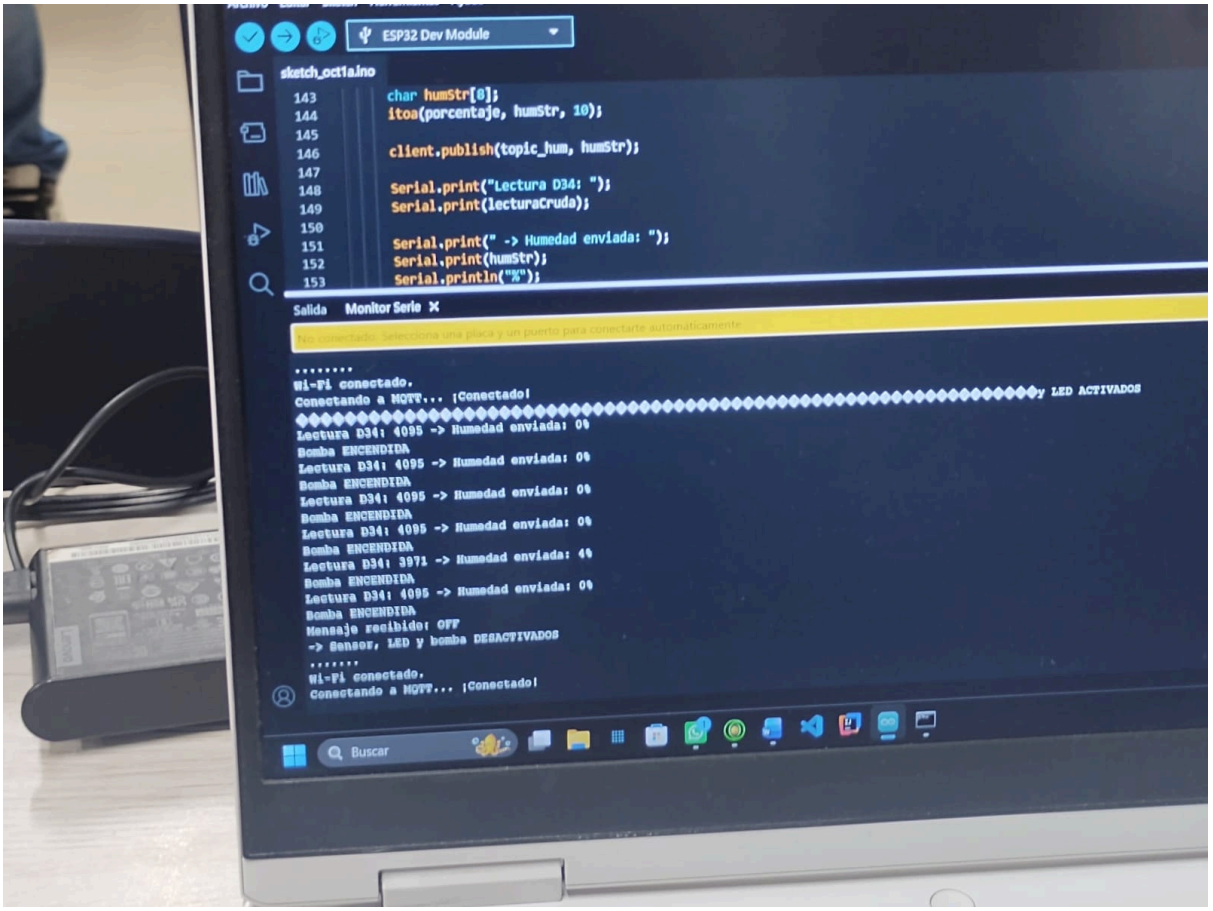
PS C:\WINDOWS\system32> Get-NetFirewallRule -DisplayName "Mosquitto MQTT TLS 8883"
```

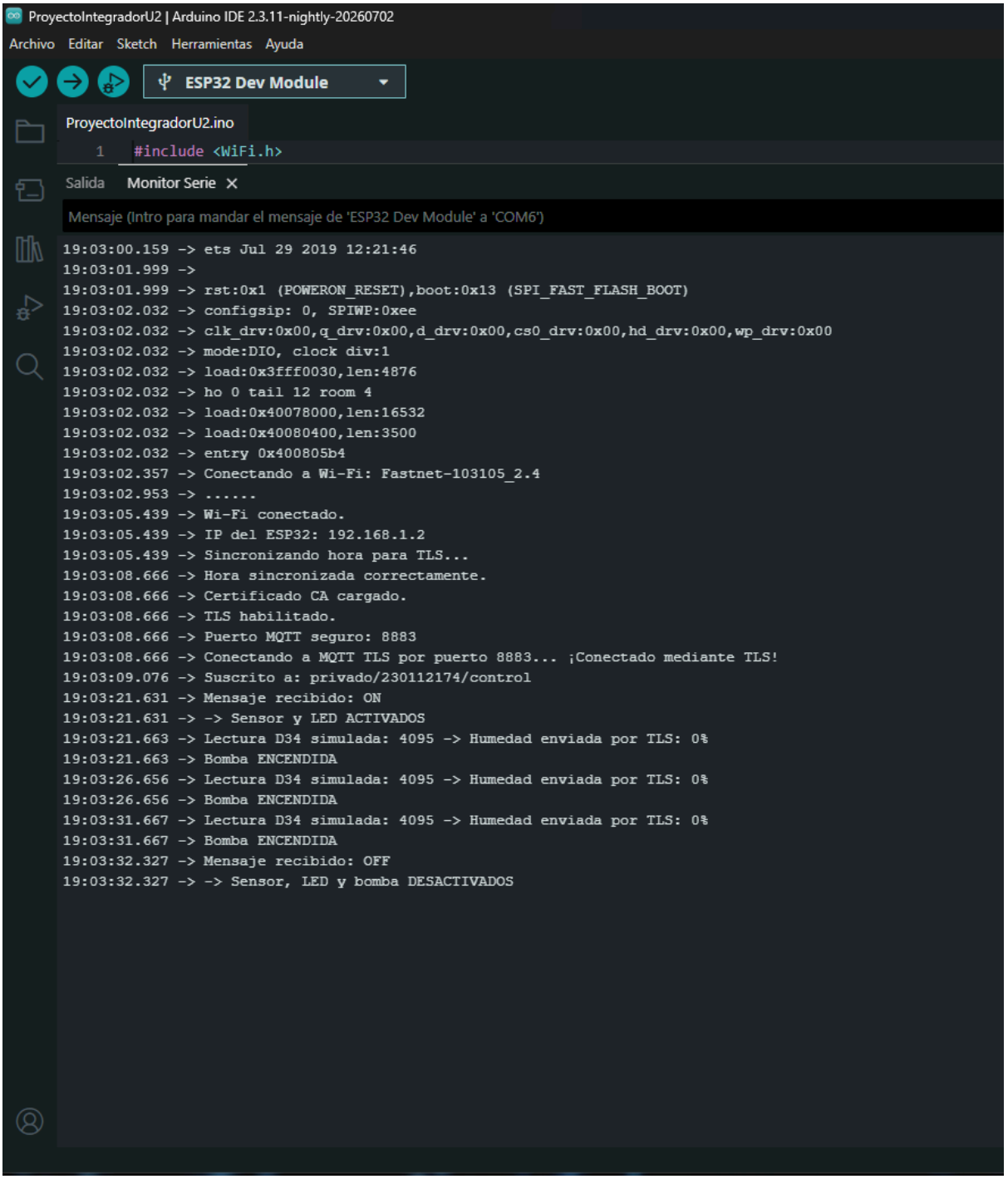
Pruebas

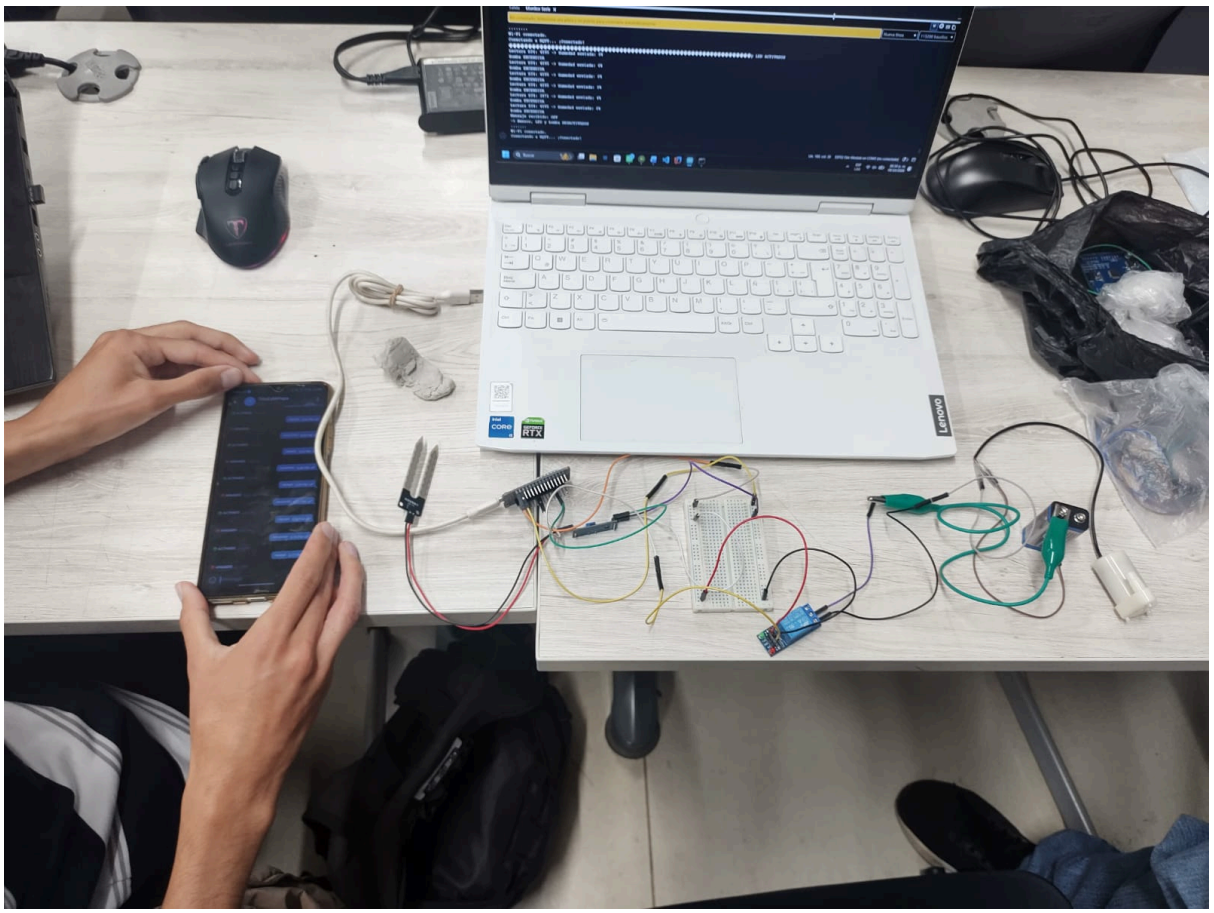

Editar nodo mqtt out > Editar nodo mqtt-broker
Eliminar
Cancelar
Actualizar
Propiedades
Nombre
ProIntegrador
Conexión
Seguridad
Mensajes
Servidor
192.168.1.6
Puerto
8883
Conectar automáticamente
Utilizar TLS
Certificado
Protocolo
MQTT V3.1.1
ID Cliente
Dejar en blanco para auto generado
Mantener activo
60
Sesión
Usar sesión limpia
Habilitado
4
En todos los flujos

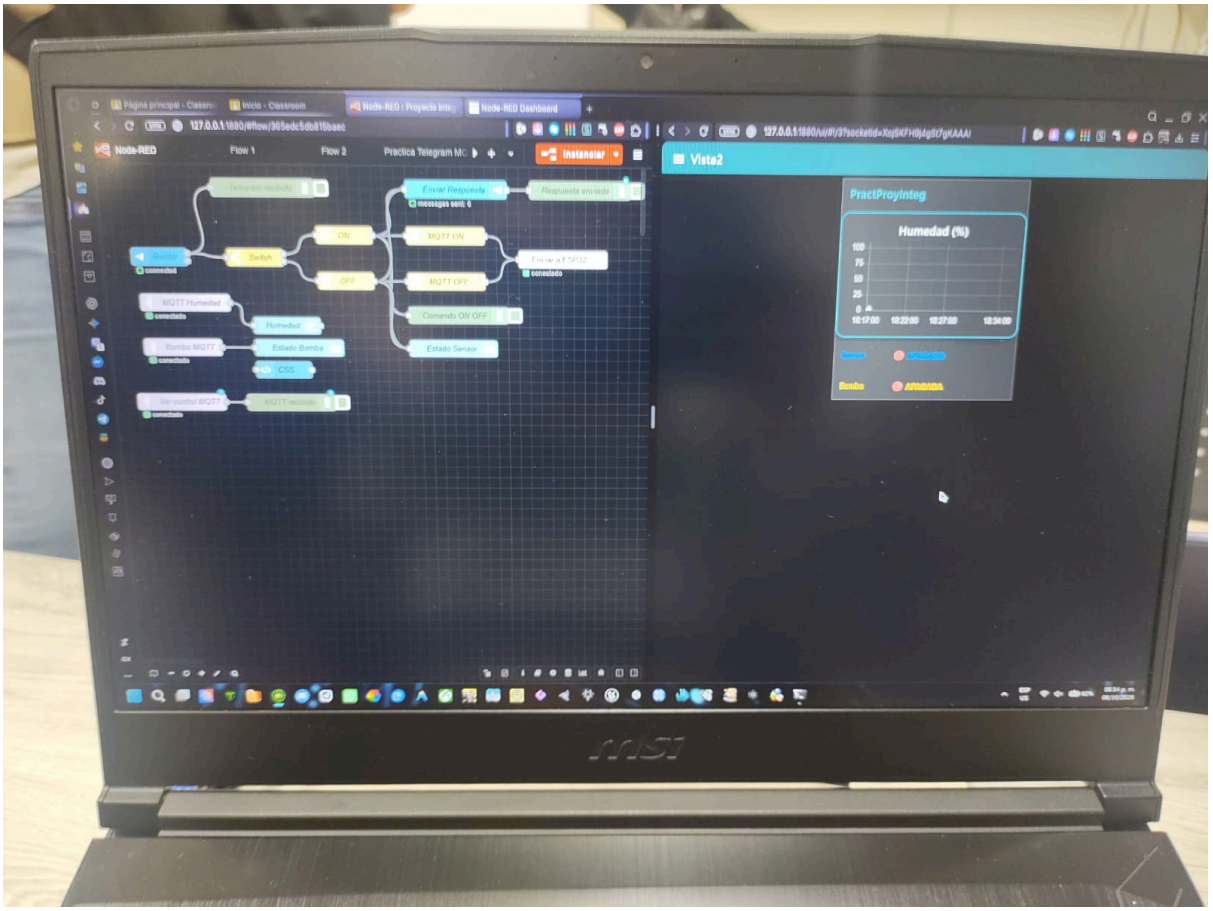

Editar nodo mqtt out > Editar nodo mqtt-broker > Editar nodo tls-config
Eliminar
Cancelar
Actualizar
Propiedades
Cert Type
Individual files
Utilizar claves y certificados de archivos locales
Certificado
Cargar
Clave Privada
Cargar
Frase de contraseña
frase de contraseña de clave privada (opc
Certificado CA
Cargar
ca.crt
Verificar el certificado del servidor
Nombre del servidor
para uso con SNI
Protocolo ALPN
para uso con ALPN
Nombre
Certificado
Habilitado
1
En todos los flujos

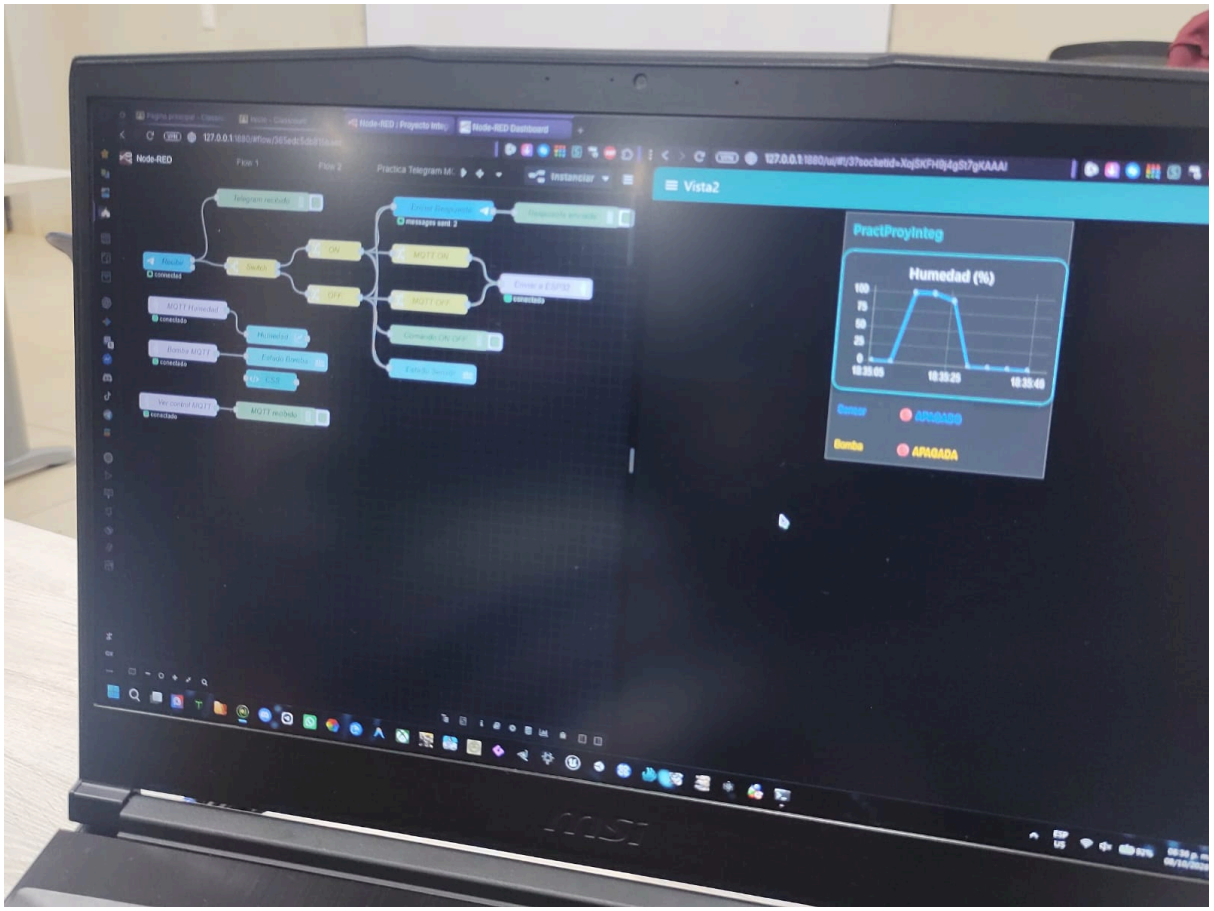

sketch_oct1a | Arduino IDE 2.3.8
Archivo Editar Sketch Herramientas Ayuda
ESP32 Dev Module
sketch_oct1a.ino
143 char humStr[8];
144 itoa(porcentaje, humStr, 10);
145
146 client.publish(topic_hum, humStr);
147
148 Serial.print("Lectura D34: ");
149 Serial.print(lecturaCruda);
150
151 Serial.print(" -> Humedad enviada: ");
152 Serial.print(humStr);
153 Serial.println("%");
Salida Monitor Serie
Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM5')
-> Sensor y LED ACTIVADOS
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 1327 -> Humedad enviada: 100%
Bomba APAGADA
Lectura D34: 1280 -> Humedad enviada: 100%
Bomba APAGADA
Lectura D34: 1636 -> Humedad enviada: 91%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Mensaje recibido: OFF
-> Sensor, LED y bomba DESACTIVADOS
Ln. 160, col. 39
Buscar

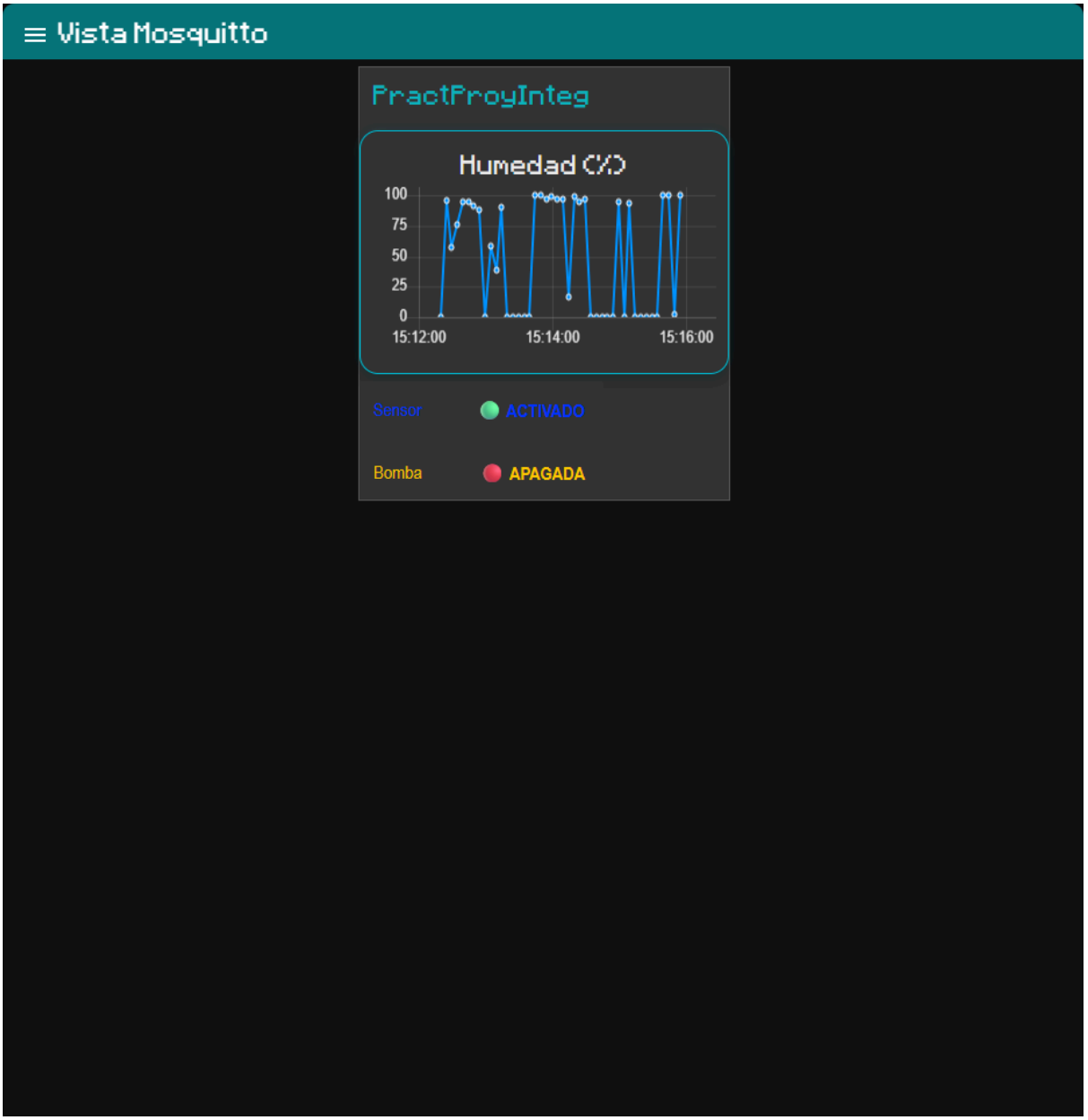

ESP32 Dev Module
sketch_oct1a.ino
143 char humStr[8];
144 itoa(porcentaje, humStr, 10);
145
146 client.publish(topic_hum, humStr);
147
148 Serial.print("Lectura D34: ");
149 Serial.print(lecturaCruda);
150
151 Serial.print(" -> Humedad enviada: ");
152 Serial.print(humStr);
153 Serial.println("%");
Salida Monitor Serie
No conectado. Selecciona una placa y un puerto para conectarte automáticamente.
.....
Wi-Fi conectado.
Conectando a MQTT... ¡Conectado!
y LED ACTIVADOS
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Lectura D34: 3971 -> Humedad enviada: 4%
Bomba ENCENDIDA
Lectura D34: 4095 -> Humedad enviada: 0%
Bomba ENCENDIDA
Mensaje recibido: OFF
-> Sensor, LED y bomba DESACTIVADOS
.....
Wi-Fi conectado.
Conectando a MQTT... ¡Conectado!
Buscar

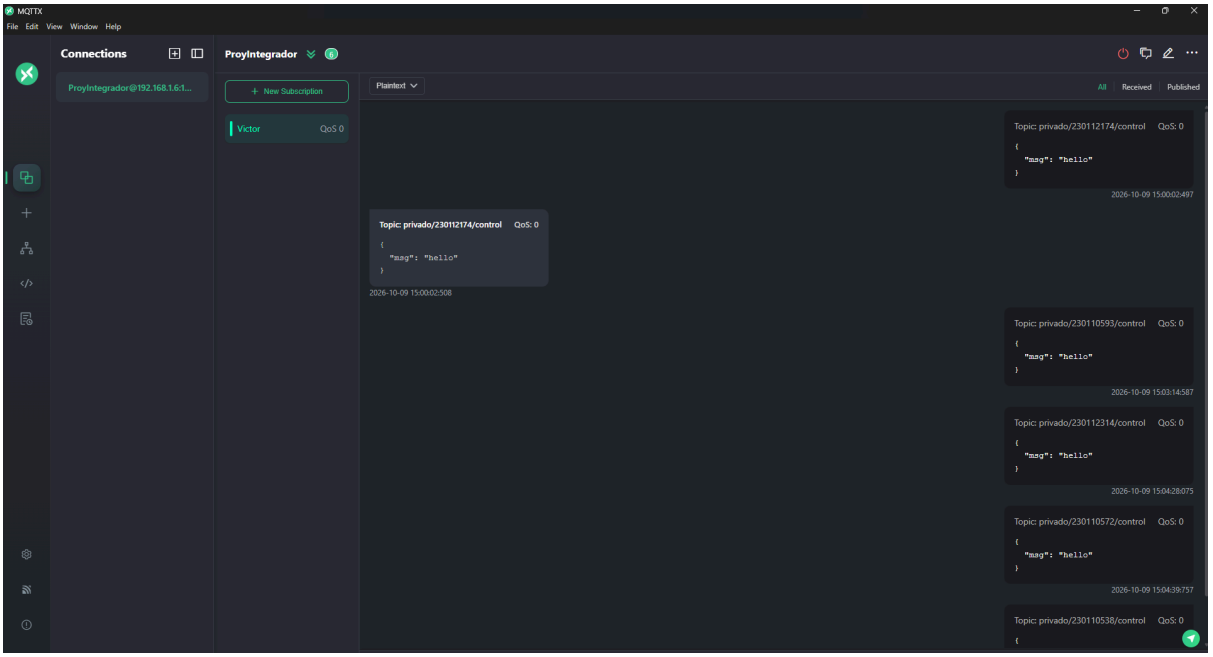

ProyectoIntegradorU2 | Arduino IDE 2.3.11-nightly-20260702
Archivo Editar Sketch Herramientas Ayuda
ESP32 Dev Module
ProyectoIntegradorU2.ino
1 #include <WiFi.h>
Salida Monitor Serie
Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM6')
19:03:00.159 -> ets Jul 29 2019 12:21:46
19:03:01.999 ->
19:03:01.999 -> rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
19:03:02.032 -> configsip: 0, SPIWP:0xee
19:03:02.032 -> clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
19:03:02.032 -> mode:DIO, clock div:1
19:03:02.032 -> load:0x3fff0030,len:4876
19:03:02.032 -> ho 0 tail 12 room 4
19:03:02.032 -> load:0x40078000,len:16532
19:03:02.032 -> load:0x40080400,len:3500
19:03:02.032 -> entry 0x400805b4
19:03:02.357 -> Conectando a Wi-Fi: Fastnet-103105_2.4
19:03:02.953 -> ......
19:03:05.439 -> Wi-Fi conectado.
19:03:05.439 -> IP del ESP32: 192.168.1.2
19:03:05.439 -> Sincronizando hora para TLS...
19:03:08.666 -> Hora sincronizada correctamente.
19:03:08.666 -> Certificado CA cargado.
19:03:08.666 -> TLS habilitado.
19:03:08.666 -> Puerto MQTT seguro: 8883
19:03:08.666 -> Conectando a MQTT TLS por puerto 8883... ¡Conectado mediante TLS!
19:03:09.076 -> Suscrito a: privado/230112174/control
19:03:21.631 -> Mensaje recibido: ON
19:03:21.631 -> -> Sensor y LED ACTIVADOS
19:03:21.663 -> Lectura D34 simulada: 4095 -> Humedad enviada por TLS: 0%
19:03:21.663 -> Bomba ENCENDIDA
19:03:26.656 -> Lectura D34 simulada: 4095 -> Humedad enviada por TLS: 0%
19:03:26.656 -> Bomba ENCENDIDA
19:03:31.667 -> Lectura D34 simulada: 4095 -> Humedad enviada por TLS: 0%
19:03:31.667 -> Bomba ENCENDIDA
19:03:32.327 -> Mensaje recibido: OFF
19:03:32.327 -> -> Sensor, LED y bomba DESACTIVADOS

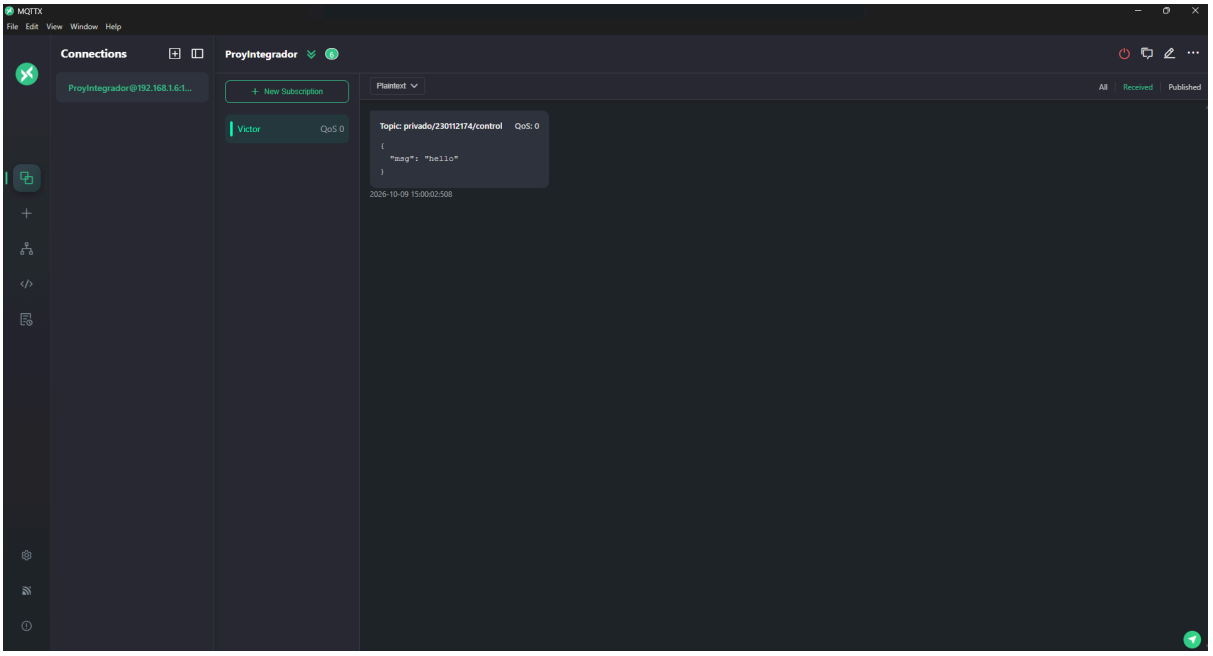

LENOVO


Node-RED
Flow 1
Flow 2
Instalar
Vista2
PractProyInteg
Humedad (%)
Bomba
APAGADA


Node-RED
Vista2
PractProyInteg
Humedad (%)
Bomba
APAGADA


Vista Mosquitto
PractProyInteg
Humedad (%)
100
75
50
25
0
15:12:00
15:14:00
15:16:00
Sensor
ACTIVADO
Bomba
APAGADA


MQTTX
File Edit View Window Help
Connections
ProyIntegrador
ProyIntegrador@192.168.1.6:1...
New Subscription
Victor QoS 0
Plaintext
All Received Published
Topic: privado/230112174/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:00:02:497
Topic: privado/230112174/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:00:02:508
Topic: privado/230110593/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:03:14:587
Topic: privado/230112314/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:04:28:075
Topic: privado/230110572/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:04:39:757
Topic: privado/230110538/control QoS: 0
{


MQTTX
File Edit View Window Help
Connections
ProyIntegrador
ProyIntegrador@192.168.1.6:1...
New Subscription
Victor QoS 0
Plaintext
All Received Published
Topic: privado/230112174/control QoS: 0
{
"msg": "hello"
}
2026-10-09 15:00:02:508

Preguntas

ALVAREZ DONOSA FABIAN MISAEL-Ingeniero de Hardware y Sensores (Firmware / Hardware) :

¿Cuáles fueron los principales desafíos técnicos que enfrentaste en el módulo correspondiente a tu rol?

Por la parte de la conexión fue complicado hacer que todo se mantuviera en su sitio el tiempo posible además de la movilidad que esto enfrentaba a la hora de desmontar y volver a armarlo según el entorno lo requiere.

¿Cómo reacciona tu módulo si el microcontrolador pierde alimentación eléctrica o si la Raspberry Pi se desconecta de la red durante la operación ?

No funcionaria la parte principal encargada de suministrar la potencia a la bomba para que pudiese desarrollarse el actuador principal de la bomba, si en cambio el raspberry pi se desconectará no podría llegar instrucción alguna al actuador.

NAVA MARTINEZ GAEL SALVADOR-Ingeniero de Comunicaciones e Infraestructura (Broker / Network)-:

¿Cuáles fueron los principales desafíos técnicos que enfrentaste en el módulo correspondiente a tu rol?

El desafío más crítico fue implementar la capa de seguridad TLS en Mosquitto, lo cual exigió la correcta generación de certificados autofirmados y la estricta alineación de direcciones IP en el archivo SAN. Además, el enrutamiento bidireccional desde la máquina anfitrión requirió excepciones precisas en el firewall y garantizar que el ESP32 sincronizara su reloj por NTP para evitar que los paquetes cifrados fueran rechazados por discrepancia de tiempo o identidad.

¿Cómo reacciona tu módulo si el microcontrolador pierde alimentación eléctrica o si la Raspberry Pi se desconecta de la red durante la operación?

Si el ESP32 pierde energía de forma abrupta, el servidor Mosquitto ejecuta el mecanismo de *Last Will and Testament* (LWT) para notificar inmediatamente a Node-RED sobre la desconexión del nodo y salvaguardar el estado de la bomba. Por otro lado, si el servidor central se desconecta de la red, el broker deja de enrutar mensajes momentáneamente, pero el código del microcontrolador está diseñado para entrar en un ciclo de reconexión autónomo hasta que el enlace TCP seguro se restablezca.

PEREZ CORONADO VICTOR HUGO-Ingeniero de Software y Orquestación (Node-RED / Dashboard / Bot):

¿Cuáles fueron los principales desafíos técnicos que enfrentaste en el módulo correspondiente a tu rol?

El mayor reto fue estructurar la lógica de los flujos para sincronizar el estado de la bomba entre el hardware y el dashboard sin crear ciclos de retroalimentación infinita. Además, el procesamiento y parseo de los payloads JSON fue complejo, especialmente al integrar la API del bot para asegurar que las alertas de humedad se enviaran en tiempo real sin saturar la aplicación.

¿Cómo reacciona tu módulo si el microcontrolador pierde alimentación eléctrica o si la Raspberry Pi se desconecta de la red durante la operación?

Ante la caída eléctrica del microcontrolador, el dashboard detecta la interrupción del flujo de datos, deshabilitando los botones de la bomba y emitiendo una alerta visual para el usuario. Si el servidor pierde la red, la interfaz advierte inmediatamente el estado de desconexión y Node-RED retiene el último estado lógico en memoria, garantizando una recuperación limpia al restablecerse el servicio.

DE ANDA GUTIERREZ ISMAEL DE JESÚS -Ingeniero de Integración y Calidad (QA / Testing & Field Test):

¿Cuáles fueron los principales desafíos técnicos que enfrentaste en el módulo correspondiente a tu rol?

El mayor desafío consistió en diseñar casos de prueba exhaustivos para calibrar los umbrales físicos del sensor de humedad y medir el impacto de la latencia introducida por el cifrado TLS en el actuador. Además, fue complejo simular escenarios de inyección de fallos y pérdida de paquetes para certificar que el estado físico de la bomba siempre coincidiera con los indicadores visuales sin generar falsos positivos.

¿Cómo reacciona tu módulo si el microcontrolador pierde alimentación eléctrica o si la Raspberry Pi se desconecta de la red durante la operación?

Las pruebas de contingencia validaron que, ante un corte eléctrico del microcontrolador, el hardware del relé entra en estado seguro (normalmente abierto) apagando la bomba físicamente para prevenir derrames. Si la Raspberry Pi pierde conexión, nuestras pruebas de campo confirmaron que el ESP32 mantiene su ciclo de evaluación interno, controlando la bomba de manera autónoma hasta que se restablezca la telemetría.

PADILLA GUTIERREZ OSVALDO -Ingeniero de Integración y Calidad (QA / Testing & Field Test):

¿Cuáles fueron los principales desafíos técnicos que enfrentaste en el módulo correspondiente a tu rol?

La mayor complicación radicó en estructurar pruebas minuciosas destinadas a ajustar las lecturas críticas del detector hídrico, evaluando asimismo el retardo derivado de la protección TLS sobre el elemento de activación. Igualmente, representó un reto recrear condiciones de falla en la transmisión para comprobar la correspondencia exacta entre el estado real del sistema y las representaciones en pantalla.

¿Cómo reacciona tu módulo si el microcontrolador pierde alimentación eléctrica o si la Raspberry Pi se desconecta de la red durante la operación?

Los ensayos de fallo demostraron que, si la unidad principal se queda sin energía, la bobina conmutadora se deshabilita para cortar el flujo de agua mecánicamente. Por otra parte, al interrumpirse el enlace con el servidor, la tarjeta lógica conserva su rutina de análisis propio, regulando el paso de fluido independientemente hasta reanudar el envío de datos.

Links

Video

PruebaFuncionamiento.mp4

Repositorio

https://github.com/dev-6a31/proyecto-sensorHumedad